

Ingeniería del software de componentes y sistemas distribuidos

PR2 - De la creación a la interconexión de microservicios

Índice

Ejercicio 2.....	3
Courses collection.....	3
POST /courses.....	3
GET /courses.....	4
PUT /courses/{id}.....	5
GET /courses/{id}.....	6
POST /courses/{courseId}/open-enrollment.....	7
POST /courses/{courseId}/enroll.....	9
GET /courses/{courseId}/enrollments.....	11
GET /courses/{courseId}/enrolled-students.....	12
POST /courses/{courseId}/close-enrollment.....	13
POST /courses/{courseId}/close-grades.....	15
POST /courses/{courseId}/close.....	16
Microcredential collection.....	19
POST /microcredentials/{courseId}/request.....	19
GET /microcredentials/pending.....	20
PUT /microcredentials/{microcredentialId}/reject.....	20
PUT /microcredentials/{microcredentialId}/approve.....	21
GET /microcredentials/{microcredentialId}.....	22

Ejercicio 2

NOTA: Voy a considerar este ejercicio también como parte de la validación del ejercicio 1, el cual pide demostrar que el proyecto funciona correctamente.

Courses collection

POST /courses

The screenshot shows a REST client interface with a sidebar on the left containing a collection tree. The main panel displays a POST request to `{{baseUrl}}/courses` with a JSON body. The response is a 201 Created status with a 74 ms response time and 214 B body size. The response body is a JSON object representing a course.

```
1 {
2   "description": "Nuevo curso de ciberseguridad",
3   "duration": 900,
4   "enrollmentEndDate": "2026-12-31T23:59:05.103Z",
5   "enrollmentStartDate": "2026-01-01T23:59:05.103Z",
6   "instructor": "instructor1@uoc.edu",
7   "language": "Español",
8   "location": "Málaga",
9   "methology": "Metodología 323",
10  "mode": "Presencial",
11  "objectives": "Pentesting, análisis forense",
12  "price": 9900,
13  "title": "Ciberseguridad en entornos TI"
```

Happy path (instructor = instructor1@uoc.edu existe y es instructor)

OpenAPI definition > courses > create Course

POST `{{baseUrl}}/courses` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

raw JSON Schema Beautify

```

1 {
2   "description": "Nuevo curso de ciberseguridad",
3   "duration": 900,
4   "enrollmentEndDate": "2026-12-31T23:59:05.103Z",
5   "enrollmentStartDate": "2026-01-01T23:59:05.103Z",
6   "instructor": "josecarlos@uoc.edu",
7   "language": "Español",
8   "location": "Málaga",
9   "methology": "Metodología 323",
10  "mode": "Presencial",
11  "objectives": "Pentesting, análisis forense",
12  "price": 9900,
13  "title": "Ciberseguridad en entornos TI"
}

```

Body 500 Internal Server Error • 10 ms • 331 B • Save Response

```

1 {
2   "timestamp": "2026-05-19T15:50:37.779+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Could not fetch instructor with email josecarlos@uoc.edu",
6   "path": "/courses"
7 }

```

Unhappy path (instructor = josecarlos@uoc.edu no existe)

OpenAPI definition > courses > create Course

POST `{{baseUrl}}/courses` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

raw JSON Schema Beautify

```

1 {
2   "description": "Nuevo curso de ciberseguridad",
3   "duration": 900,
4   "enrollmentEndDate": "2026-12-31T23:59:05.103Z",
5   "enrollmentStartDate": "2026-01-01T23:59:05.103Z",
6   "instructor": "alumno1@uoc.edu",
7   "language": "Español",
8   "location": "Málaga",
9   "methology": "Metodología 323",
10  "mode": "Presencial",
11  "objectives": "Pentesting, análisis forense",
12  "price": 9900,
13  "title": "Ciberseguridad en entornos TI"
}

```

Body 500 Internal Server Error • 21 ms • 319 B • Save Response

```

1 {
2   "timestamp": "2026-05-19T15:47:43.735+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "The user alumno1@uoc.edu is not instructor !",
6   "path": "/courses"
7 }

```

Unhappy path (instructor = alumno1@uoc.edu no es STUDENT)

GET /courses

Overview POST cre. POST ope. POST enr. GET get C. GET find C. Createc. + No environment

OpenAPI definition > courses > find Courses Save Share

GET {{baseUrl}}/courses Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK 8 ms 1.81 KB Save Response

{ } JSON Preview Visualize

```
48      "duration": 100,  
49      "language": "English",  
50      "location": "on the net",  
51      "status": "ACTIVE"  
52    },  
53    ],  
54    {  
55      "id": 4,  
56      "instructor": "instructor1@uoc.edu",  
57      "enrollment": null,  
58      "title": "Ciberseguridad en entornos TI",  
59      "description": "Nuevo curso de ciberseguridad",  
60      "enrollmentStartDate": "2026-01-01T23:59:05.103+00:00",  
61      "enrollmentEndDate": "2026-12-31T23:59:05.103+00:00",  
62      "mode": "Presencial",  
63      "price": 9900,  
64      "objectives": "Pentesting, análisis forense",  
65      "methodology": "Metodologia 323",  
66      "duration": 900,  
67      "language": "Español",  
68      "location": "Málaga",  
69      "status": "DRAFT"  
70    }  
71  ]
```

PUT /courses/{id}

The screenshot displays a REST client interface with a sidebar on the left and a main workspace on the right. The sidebar, titled 'COLLECTIONS', lists various API endpoints under 'OpenAPI definition' and 'courses'. The main workspace shows a PUT request to the endpoint `{{baseUrl}}/courses/:courseId`. The request body is a JSON object with the following fields: `description`, `duration`, `enrollmentEndDate`, `enrollmentStartDate`, `instructor`, `language`, `location`, `methology`, `mode`, `objectives`, `price`, and `title`. The response status is 200 OK, and the response body is a single character '1'.

Overview | POST cre. | POST ope. | POST enr. | GET get C. | GET find Cc. | PUT mod. | + | No environment

OpenAPI definition > courses > {courseId} > modify Course Details

PUT | `{{baseUrl}}/courses/:courseId` | Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookies

raw | JSON | Schema | Beautify

```
1 {
2   "description": "Nuevo curso de ciberseguridad",
3   "duration": 800,
4   "enrollmentEndDate": "2026-12-31T23:59:05.103Z",
5   "enrollmentStartDate": "2026-01-01T23:59:05.103Z",
6   "instructor": "instructor@uoc.edu",
7   "language": "Español",
8   "location": "Málaga",
9   "methology": "Metodologia 323",
10  "mode": "Presencial",
11  "objectives": "Pentesting, análisis forense",
12  "price": 9900,
13  "title": "Ciberseguridad en entornos TI"
14 }
```

Body | 200 OK | 30 ms | 123 B | Save Response

Raw | Preview | Visualize | 1

GET /courses/{id}

The screenshot shows the Postman interface with a collection named 'My Collection' containing an 'OpenAPI definition' folder. Inside this folder, there is a 'courses' folder, which contains a '{courseId}' folder. The '{courseId}' folder has several endpoints, including 'GET get Course By Id'. The 'GET get Course By Id' endpoint is selected, and the URL is set to 'GET {{baseUrl}}/courses/:courseId'. The 'Send' button is clicked, and the response is displayed in the 'Body' tab. The response is a JSON object with the following fields: 'id' (4), 'instructor' (instructor@uoc.edu), 'enrollment' (null), 'title' (Ciberseguridad en entornos TI), 'description' (Nuevo curso de ciberseguridad), 'enrollmentStartDate' (2026-01-01T23:59:05.103+00:00), 'enrollmentEndDate' (2026-12-31T23:59:05.103+00:00), 'mode' (Presencial), 'price' (9900), 'objectives' (Pentesting, análisis forense), 'methodology' (Metodología 323), 'duration' (800), 'language' (Español), 'location' (Málaga), and 'status' (DRAFT). The status is highlighted in blue.

Overview POST cre POST ope POST enr GET get GET find Co PUT mod + No environment

OpenAPI definition > courses > {courseId} > get Course By Id Save Share

GET {{baseUrl}}/courses/:courseId Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Path Variables

Key	Value	Description	Bulk Edit	...
courseId *	integer 4	Description		

Body 200 OK 15 ms 600 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 4,
3   "instructor": "instructor@uoc.edu",
4   "enrollment": null,
5   "title": "Ciberseguridad en entornos TI",
6   "description": "Nuevo curso de ciberseguridad",
7   "enrollmentStartDate": "2026-01-01T23:59:05.103+00:00",
8   "enrollmentEndDate": "2026-12-31T23:59:05.103+00:00",
9   "mode": "Presencial",
10  "price": 9900,
11  "objectives": "Pentesting, análisis forense",
12  "methodology": "Metodología 323",
13  "duration": 800,
14  "language": "Español",
15  "location": "Málaga",
16  "status": "DRAFT"
17 }
```

Happy path (courseId = 4 existe)

The screenshot shows the Postman interface with the same collection and endpoint as the previous screenshot. The 'GET get Course By Id' endpoint is selected, and the URL is set to 'GET {{baseUrl}}/courses/:courseId'. The 'Send' button is clicked, and the response is displayed in the 'Body' tab. The response is a JSON object with the following fields: 'timestamp' (2026-05-19T13:50:20.018+00:00), 'status' (500), 'error' (Internal Server Error), 'message' (Course with id '100' not found), and 'path' (/courses/100). The status is highlighted in red.

Overview GET find Co GET get C PUT mod GET OK PUT modify GET OK GET get Co PUT modify GET get C + No environment

OpenAPI definition > courses > {courseId} > get Course By Id From GET OK Save Share

GET {{baseUrl}}/courses/:courseId Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Path Variables

Key	Value	Description	Bulk Edit	...
courseId	100	Description		

Body 500 Internal Server Error 17 ms 309 B

{ } JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-05-19T13:50:20.018+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Course with id '100' not found",
6   "path": "/courses/100"
7 }
```

Unhappy path (courseId = 100 no existe)

POST /courses/{courseId}/open-enrollment

The screenshot shows the Postman interface with the following details:

- Request:** POST `{{baseUrl}}/courses/{courseId}/open-enrollment`
- Path Variables:**

Key	Value	Description
courseId	4	
- Response:** 200 OK, 15 ms, 123 B
- Body:** Raw (JSON)

Happy path (abrir inscripción cuando **no** está abierta)

The screenshot shows the Postman interface with the following details:

- Request:** GET `{{baseUrl}}/courses/{courseId}`
- Path Variables:**

Key	Value	Description
courseId	4	
- Response:** 200 OK, 10 ms, 611 B
- Body:** JSON

```
1 {
2   "id": 4,
3   "instructor": "instructor1@uoc.edu",
4   "enrollment": null,
5   "title": "Ciberseguridad en entornos TI",
6   "description": "Nuevo curso de ciberseguridad",
7   "enrollmentStartDate": "2026-01-01T23:59:05.103+00:00",
8   "enrollmentEndDate": "2026-12-31T23:59:05.103+00:00",
9   "mode": "Presencial",
10  "price": 9900,
11  "objectives": "Pentesting, análisis forense",
12  "methodology": "Metodología 323",
13  "duration": 900,
14  "language": "Español",
15  "location": "Málaga",
16  "status": "ENROLLMENT_OPEN"
17 }
```

Happy path (el status cambia a ENROLLMENT_OPEN)

Overview POST cre. POST op. POST enr. GET get C. GET find Co PUT mod. + No environment

OpenAPI definition > cour... > {courseId} > open-enrollment > open Enrollment Save Share

POST {{baseUrl}}/courses/{courseId}/open-enrollment Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Path Variables

Key	Value	Description	Bulk Edit	...
courseId *	integer 4	Description		

Body 500 Internal Server Error 12 ms 332 B Save Response

{ } JSON Preview Debug with AI

```

1 {
2   "timestamp": "2026-05-19T15:56:49.272+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Enrollment is already open for course 4",
6   "path": "/courses/4/open-enrollment"
7 }

```

Unhappy path (abrir inscripción cuando está abierta)

POST /courses/{courseId}/enroll

The screenshot shows a REST client interface with a sidebar on the left containing a collection tree. The main panel displays a POST request to the endpoint `POST {{baseUrl}}/courses/{courseId}/enroll?student=user1@uoc.edu`. The request parameters are defined in two tables:

Key	Value	Description	Bulk Edit	...
student *	string	user1@uoc.edu		
Key	Value	Description		

Key	Value	Description	Bulk Edit	...
courseId *	integer	4		
Key	Value	Description		

The response body is shown in the 'Raw' tab, displaying the status `201 Created`, a response time of `22 ms`, and a body size of `128 B`. The response content is the number `1`.

Happy path (courseId = 4 y student = user1@uoc.edu existen)

The screenshot shows the same REST client interface, but with a different request parameter. The `courseId` is now `40`. The response body is shown in the 'JSON' tab, displaying a `500 Internal Server Error` with the following details:

```
1 {
2   "timestamp": "2026-05-19T16:15:01.996+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Course with id '40' not found",
6   "path": "/courses/40/enroll"
7 }
```

Unhappy path (courseId = 40 no existe)

The screenshot shows the Swagger UI interface for an API. The left sidebar displays a collection of endpoints under 'collections'. The main panel shows the 'enroll In Course' endpoint (POST) with the URL: `{{baseUrl}}/courses/{courseId}/enroll?student=inventado@uoc.edu`. The 'Params' tab is active, showing query parameters: 'student' (string, 'inventado@uoc.edu') and 'courseId' (integer, '3'). The response body shows a '500 Internal Server Error' with the following JSON:

```

{
  "timestamp": "2026-05-19T16:07:01.130+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "User with id 'inventado@uoc.edu' not found",
  "path": "/courses/3/enroll"
}

```

Unhappy path (student = inventado@uoc.edu no existe)

The screenshot shows the Swagger UI interface for an API. The left sidebar displays a collection of endpoints under 'collections'. The main panel shows the 'enroll In Course' endpoint (POST) with the URL: `{{baseUrl}}/courses/{courseId}/enroll?student=user2@uoc.edu`. The 'Params' tab is active, showing query parameters: 'student' (string, 'user2@uoc.edu') and 'courseId' (integer, '4'). The response body shows a '500 Internal Server Error' with the following JSON:

```

{
  "timestamp": "2026-05-19T17:14:10.953+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Course 4 is not open for enrollment",
  "path": "/courses/4/enroll"
}

```

Unhappy path (courseId= 4 no tiene status ENROLLMENT_OPEN)

GET /courses/{courseId}/enrollments

The screenshot shows the Postman interface with the following details:

- URL:** `{{baseUrl}}/courses/:courseId/enrollments`
- Method:** GET
- Path Variables:**

Key	Value	Description	Bulk Edit
courseId *	integer 4	Description	
- Status:** 200 OK
- Body (JSON):**

```
1 [
2   {
3     "id": 6,
4     "student": "user1@uoc.edu",
5     "enrollmentDate": "2026-05-19T16:32:12.447+00:00",
6     "qualification": 0,
7     "status": "ACTIVE",
8     "courseId": 4
9   },
10  {
11    "id": 7,
12    "student": "user2@uoc.edu",
13    "enrollmentDate": "2026-05-19T16:32:16.218+00:00",
14    "qualification": 0,
15    "status": "ACTIVE",
16    "courseId": 4
17  }
18 ]
```

Happy path (courseId = 4 existe)

The screenshot shows the Postman interface with the following details:

- URL:** `{{baseUrl}}/courses/:courseId/enrollments`
- Method:** GET
- Path Variables:**

Key	Value	Description	Bulk Edit
courseId *	integer 40	Description	
- Status:** 500 Internal Server Error
- Body (JSON):**

```
1 {
2   "timestamp": "2026-05-19T16:33:11.031+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Course with id '40' not found",
6   "path": "/courses/40/enrollments"
7 }
```

Unhappy path (courseId = 40 no existe)

GET /courses/{courseId}/enrolled-students

The screenshot shows the Postman interface with the following details:

- URL:** `{{baseUrl}}/courses/:courseId/enrolled-students`
- Method:** GET
- Path Variables:**

Key	Value	Description	Bulk Edit	...
courseId	integer	4		
- Response:** 200 OK, 10 ms, 197 B. The response body is a JSON array:

```
[
  [
    "user1@uoc.edu",
    "user2@uoc.edu"
  ]
]
```

Happy path (courseId = 4 existe)

The screenshot shows the Postman interface with the following details:

- URL:** `{{baseUrl}}/courses/:courseId/enrolled-students`
- Method:** GET
- Path Variables:**

Key	Value	Description	Bulk Edit	...
courseId	integer	40		
- Response:** 500 Internal Server Error, 9 ms, 325 B. The response body is a JSON object:

```
{
  "timestamp": "2026-05-19T16:34:02.593+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Course with id '40' not found",
  "path": "/courses/40/enrolled-students"
}
```

Happy path (courseId = 40 no existe)

POST /courses/{courseId}/close-enrollment

The screenshot shows the Postman interface with a collection named 'close-enrollment' selected. The request is a POST to the endpoint `{{baseUrl}}/courses/{courseId}/close-enrollment`. The 'Path Variables' section shows `courseId` with a value of `4`. The response is a `200 OK` status with a response time of 15 ms and a body size of 123 B. The response body is `1`.

Happy path (courseId = 4 existe)

The screenshot shows the Postman interface with a collection named 'get Course By Id' selected. The request is a GET to the endpoint `{{baseUrl}}/courses/{courseId}`. The 'Path Variables' section shows `courseId` with a value of `4`. The response is a `200 OK` status with a response time of 7 ms and a body size of 602 B. The response body is a JSON object:

```
1 {
2   "id": 4,
3   "instructor": "instructor1@uoc.edu",
4   "enrollment": null,
5   "title": "Ciberseguridad en entornos TI",
6   "description": "Nuevo curso de ciberseguridad",
7   "enrollmentStartDate": "2026-01-01T23:59:05.103+00:00",
8   "enrollmentEndDate": "2026-12-31T23:59:05.103+00:00",
9   "mode": "Presencial",
10  "price": 9900,
11  "objectives": "Pentesting, análisis forense",
12  "methodology": "Metodología 323",
13  "duration": 900,
14  "language": "Español",
15  "location": "Málaga",
16  "status": "ACTIVE"
17 }
```

Happy path (el status de courseId = 4 es ACTIVE)

The screenshot shows a REST client interface with a sidebar on the left listing API collections. The main panel displays a POST request to `{{baseUrl}}/courses/:courseId/close-enrollment`. The request body is empty. The response is a 500 Internal Server Error with the following JSON body:

```
{
  "timestamp": "2026-05-19T16:35:24.156+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Enrollment is not open for course 4",
  "path": "/courses/4/close-enrollment"
}
```

The error message indicates that the enrollment for course 4 is not open, which is the 'unhappy path' described in the caption.

Unhappy path (no se puede cerrar un periodo de matriculación que ya está cerrado)

POST /courses/{courseId}/close-grades

The screenshot shows the Postman interface with a collection named 'close Grade Reports' selected. The request is a POST to the endpoint `{{baseUrl}}/courses/{courseId}/close-grades`. The path variable `courseId` is set to the value `4`. The response is a `200 OK` status with a response time of 25 ms and a body size of 123 B. The response body is empty, indicated by the number '1'.

Happy path (courseId = 4 existe)

The screenshot shows the Postman interface with the same collection and endpoint. The path variable `courseId` is now set to the value `40`. The response is a `500 Internal Server Error` status with a response time of 40 ms and a body size of 320 B. The response body is a JSON object:

```
1 {
2   "timestamp": "2026-05-19T16:42:12.649+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "message": "Course with id '40' not found",
6   "path": "/courses/40/close-grades"
7 }
```

Unhappy path (courseId = 40 no existe)

POST /courses/{courseId}/close

The screenshot shows the Swagger UI for the endpoint `POST /courses/{courseId}/close`. The sidebar on the left lists various collections, including `POST close Course`. The main panel displays the endpoint details, including the method `POST`, the path `{{baseUrl}} /courses/ :courseId /close`, and the response status `200 OK`. The response body is shown as `1 true`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Key	Value	Description	Bulk Edit	...
courseId *	integer	4		

Body: `1 true`

Happy path (courseId = 4 existe)

The screenshot shows the Swagger UI for the endpoint `GET /courses/{courseId}/enrollments`. The sidebar on the left lists various collections, including `GET get Enrollments By Course`. The main panel displays the endpoint details, including the method `GET`, the path `{{baseUrl}} /courses/ :courseId /enrollments`, and the response status `200 OK`. The response body is shown as a JSON array:

```
1 [
2   {
3     "id": 6,
4     "student": "user2@uoc.edu",
5     "enrollmentDate": "2026-05-19T16:49:54.214+00:00",
6     "qualification": 0,
7     "status": "CLOSED",
8     "courseId": 4
9   }
10 ]
```

Happy path (el estado de las matrículas cambia a CLOSED)

The screenshot shows a REST client interface with a sidebar on the left containing a collection of API endpoints. The main panel displays the details for the 'close Course' endpoint, which is a POST request to the path `{{baseUrl}}/courses/:courseId/close`. The 'Params' tab is active, showing a path variable `courseId` with a value of `40` and a type of `integer`. The 'Body' tab shows a JSON response indicating a 500 Internal Server Error.

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Path Variables

Key	Value	Description	Bulk Edit	...
courseId *	integer 40	Description		

Body

500 Internal Server Error • 12 ms • 313 B • Save Response

```
{
  "timestamp": "2026-05-19T16:50:52.481+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Course with id '40' not found",
  "path": "/courses/40/close"
}
```

Unhappy path (courseId = 40 no existe)

Microcredential collection

POST /microcredentials/{courseId}/request

The screenshot shows a REST client interface with a sidebar on the left containing a tree of collections. The main panel displays a POST request to the endpoint `{{baseUrl}}/microcredentials/courses/:courseId/request`. The 'Params' tab is active, showing a path variable `courseId` with a value of `4` and type `integer`. The response is shown in the 'Body' tab as `{}` JSON with a status of `200 OK`, a response time of `380 ms`, and a body size of `168 B`. The response body is `true`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Key	Value	Description	Bulk Edit	...
courseId *	integer	4		Description

Body: `{}` JSON, `200 OK`, `380 ms`, `168 B`. Save Response. `true`

Happy path (courseId = 4 existe y está en estado CLOSED)

The screenshot shows the same REST client interface, but with the path variable `courseId` set to `3`. The response is shown in the 'Body' tab as `{}` JSON with a status of `200 OK`, a response time of `13 ms`, and a body size of `169 B`. The response body is `false`.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Key	Value	Description	Bulk Edit	...
courseId *	integer	3		Description

Body: `{}` JSON, `200 OK`, `13 ms`, `169 B`. Save Response. `false`

Unhappy path (courseId = 3 tiene status ACTIVE)

GET /microcredentials/pending

The screenshot shows a REST client interface with a sidebar on the left containing a collection of API endpoints. The main panel displays a GET request to `{{baseUrl}}/microcredentials/pending` which has been successfully executed, returning a 200 OK status with a response time of 78 ms and a body size of 469 B. The response body is shown in JSON format, containing two objects representing pending microcredential requests.

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

```
1 [
2   {
3     "id": 1,
4     "submitDate": "2026-05-19T18:07:47.291+00:00",
5     "assignmentDate": "2026-05-19T18:07:47.291+00:00",
6     "status": "REQUESTED",
7     "content": "",
8     "enrollment": 6
9   },
10  {
11    "id": 2,
12    "submitDate": "2026-05-19T18:07:47.291+00:00",
13    "assignmentDate": "2026-05-19T18:07:47.291+00:00",
14    "status": "REQUESTED",
15    "content": "",
16    "enrollment": 7
17  }
18 ]
```

PUT /microcredentials/{microcredentialId}/reject

The screenshot displays a REST client interface with the following components:

- Left Sidebar (COLLECTIONS):**
 - GET get Enrollments By Course (OK)
 - enrolled-students
 - GET get Enrolled Students (OK)
 - GET get Course By Id (OK)
 - PUT modify Course Details (OK)
 - GET find Courses (OK)
 - POST create Course (Created)
 - OpenAPI definition Copy
 - microcredentials
 - courses
 - {courseId}
 - request
 - POST request Course Microcredential (OK)
 - {microcredentialId}
 - reject
 - PATCH reject Pending Microcredential (selected)

- Main Area:**
- Method:** PATCH
- URL:** {{baseUrl}}/microcredentials/:microcredentialId/reject
- Params:** microcredentialId (Integer, Value: 1)
- Body:** 1
- Response:** 200 OK, 31 ms, 123 B

PUT /microcredentials/{microcredentialId}/approve

The screenshot displays a REST client interface with a sidebar on the left containing a 'COLLECTIONS' tree. The main panel shows a PATCH request configuration for the endpoint `{{baseUri}}/microcredentials/:microcredentialId/approve`. The 'Params' tab is active, showing a path variable `microcredentialId` with a value of `2`. The response status is `200 OK` with a response time of `14 ms` and a size of `123 B`. The response body is currently empty.

COLLECTIONS

- GET get Course By Id
e.g. OK
- PUT modify Course Details
e.g. OK
- GET find Courses
e.g. OK
- POST create Course
e.g. Created
- OpenAPI definition Copy
- microcredentials
 - courses
 - {courseId}
 - request
 - POST request Course Microcredential
e.g. OK
 - {microcredentialId}
 - reject
 - PATCH reject Pending Microcredential
e.g. OK
 - approve
 - PATCH approve Pending Microcredential
e.g. OK
 - get Microcredential By Id
GET

Request Configuration:

- Method: PATCH
- URL: `{{baseUri}}/microcredentials/:microcredentialId/approve`
- Params:

Key	Value	Description	Bulk Edit	...
microcredentialId	integer 2			

Response: 200 OK • 14 ms • 123 B

Body: Raw (1)

GET /microcredentials/{microcredentialId}

The screenshot shows a REST client interface with a sidebar on the left containing a collection of endpoints. The main panel displays a GET request to the endpoint `{{baseUri}}/microcredentials/{microcredentialId}` with the path variable `microcredentialId` set to `1`. The response is a 200 OK status with a response time of 12 ms and a body size of 314 B. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "submitDate": "2026-05-19T18:07:47.291+00:00",
4   "assignmentDate": "2026-05-19T18:07:47.291+00:00",
5   "status": "REJECTED",
6   "content": "",
7   "enrollment": 6
8 }
```

Microcredencial rechazada

The screenshot shows the same REST client interface, but with the path variable `microcredentialId` set to `2`. The response is a 200 OK status with a response time of 6 ms and a body size of 313 B. The response body is a JSON object:

```
1 {
2   "id": 2,
3   "submitDate": "2026-05-19T18:07:47.291+00:00",
4   "assignmentDate": "2026-05-19T18:07:47.291+00:00",
5   "status": "GRANTED",
6   "content": "",
7   "enrollment": 7
8 }
```

Microcredencial aprobada